

Quaternions Are Not That Scary

Rotations, Ruby, and a little bit of Hamilton

Ruby Nights Auckland

DragonRuby demo

Why this talk exists

Michael showed a **Three.js** construction-site visualisation.

He mentioned **quaternions** as part of the 3D rotation engine.

The room giggled nervously.

I said:

They are not too scary if viewed the right way.

So now I owe you a talk.

Tonight's promise

Not a full algebra lecture.

Not a derivation of every formula.

Instead:

| Quaternions are a practical language for 3D orientation.

And we will use them from **Ruby**.

The recurring pattern

Mathematics often grows like this:

1. We have a useful number system.
2. We meet an equation or operation it cannot handle.
3. We extend the system.
4. We gain power.
5. We sometimes lose a familiar property.

Natural numbers

We start with:

1, 2, 3, 4, ...

They are excellent for counting.

But the equation

$$x + 2 = 1$$

has no solution in natural numbers.

Integers

Add zero and negatives:

$$\dots, -2, -1, 0, 1, 2, \dots$$

Now equations like

$$x + a = b$$

always have an integer solution.

We paid a small conceptual price: numbers can now be “less than nothing”.

Rational numbers

But multiplication creates another problem:

$$3x = 2$$

No integer solution.

So we add fractions:

$$\frac{2}{3}, \frac{5}{7}, -\frac{11}{4}$$

Now division by non-zero numbers works.

Real numbers

The rationals still have gaps.

The diagonal of a unit square has length:

$$\sqrt{2}$$

It is not rational.

So we complete the number line and obtain the **real numbers**.

Complex numbers

The real numbers still cannot solve:

$$x^2 = -1$$

So we add a new number:

$$i^2 = -1$$

and obtain:

$$a + bi$$

What we gained

Complex numbers are not just “numbers with imaginary bits”.

They give us a complete algebraic world:

| Every non-constant polynomial with complex coefficients has a complex root.

From the equation-solving perspective, this is the end of the story.

But geometry continues

Real numbers live on a line.

Complex numbers live on a plane.

And complex numbers of length one rotate that plane:

$$e^{i\theta} = \cos \theta + i \sin \theta$$

Multiplication becomes rotation.

A tempting question

If complex numbers rotate the plane so beautifully...

Can we build something similar for **3D space**?

Maybe a number like:

$$a + bi + cj$$

Hamilton tried.

It did not work.

Hamilton's jump

The breakthrough was not three components.

It was four:

$$q = w + xi + yj + zk$$

with

$$i^2 = j^2 = k^2 = ijk = -1$$

These are the **quaternions**.

The price

Quaternions preserve many familiar rules.

But multiplication is no longer commutative:

$$ij = k$$

while

$$ji = -k$$

The order matters.

This is not a bug

3D rotations themselves do not commute.

Try this with a book:

1. Rotate around the x-axis.
2. Then rotate around the y-axis.

Now reverse the order.

You get a different orientation.

The algebra is telling the truth.

What is a quaternion?

A quaternion has a scalar part and a vector part:

$$q = (w, \mathbf{v})$$

or

$$q = w + xi + yj + zk$$

For rotations, we mostly care about **unit quaternions**.

Axis and angle

A 3D rotation is naturally described by:

- an axis $\mathbf{u}$
- an angle θ

The corresponding unit quaternion is:

$$q = \cos \frac{\theta}{2} + \mathbf{u} \sin \frac{\theta}{2}$$

That half-angle is important.

Rotating a point

Treat a 3D point as a “pure” quaternion:

$$p = 0 + xi + yj + zk$$

Then rotate it by:

$$p' = qpq^{-1}$$

This is the whole trick.

The Ruby version

```
class Quaternion
  attr_reader :w, :x, :y, :z

  def initialize(w, x, y, z)
    @w = w.to_f
    @x = x.to_f
    @y = y.to_f
    @z = z.to_f
  end
end
```

A quaternion is just four floating-point numbers.

The magic is in multiplication.

Quaternion multiplication

```
def *(other)
  Quaternion.new(
    w*other.w - x*other.x - y*other.y - z*other.z,
    w*other.x + x*other.w + y*other.z - z*other.y,
    w*other.y - x*other.z + y*other.w + z*other.x,
    w*other.z + x*other.y - y*other.x + z*other.w
  )
end
```

Notice: the order of multiplication matters.

Axis-angle constructor

```
def self.from_axis_angle(axis, angle)
  ax, ay, az = normalize_vector(axis)
  half_angle = angle / 2.0
  scale = Math.sin(half_angle)

  new(
    Math.cos(half_angle),
    ax * scale, ay * scale, az * scale
  )
end
```

One axis. One angle. One object.

Rotating a vector

```
def rotate(vector)
  point = Quaternion.new(0, *vector)
  result = self * point * conjugate

  [result.x, result.y, result.z]
end
```

This is where algebra touches the screen.

Why DragonRuby?

DragonRuby gives us a tiny Ruby game loop:

```
module Main
  def tick(args)
    # update state
    # draw frame
  end
end
```

For this demo, we do not need a 3D engine.

We only need:

1. rotate 3D points;
2. project them to 2D;
3. draw lines.

A box is only data

```
VERTICES = [  
    [-1, -1, -1], [ 1, -1, -1],  
    [ 1,  1, -1], [-1,  1, -1],  
    [-1, -1,  1], [ 1, -1,  1],  
    [ 1,  1,  1], [-1,  1,  1]  
]  
  
EDGES = [  
    [0, 1], [1, 2], [2, 3], [3, 0],  
    [4, 5], [5, 6], [6, 7], [7, 4],  
    [0, 4], [1, 5], [2, 6], [3, 7]  
]
```


Projection

```
def project(x, y, z)
  camera = 5.0
  scale = camera / (camera - z)

  [
    640 + x * 130 * scale,
    380 + y * 130 * scale
  ]
end
```

Fake 3D can be very convincing.

Drawing the model

```
points = args.state.model.projected_vertices(  
  args.state.orientation,  
  centre_x: 640,  
  centre_y: 380  
)
```

```
BoxModel::EDGES.each do |from_index, to_index|  
  x1, y1 = points[from_index]  
  x2, y2 = points[to_index]  
  
  args.outputs.lines << {  
    x: x1, y: y1, x2: x2, y2: y2  
  }  
end
```

DragonRuby draws ordinary 2D lines.

The quaternion supplies the orientation.

One rotation, any axis

```
axis = [1, 1, 1]
angle = Kernel.tick_count * 0.02

args.state.orientation =
    Quaternion.from_axis_angle(axis, angle)
```

This is the **next implementation milestone**.

One quaternion can rotate around a diagonal axis.

Not x, then y, then z—one spatial rotation.

Live demo controls

- **X / Y / Z** — rotate $+15^\circ$ around world axes
- **J / K / L** — rotate -15° around world axes
- **R** — reset to identity
- Compare **R, X $\times$ 6, Y $\times$ 6** with **R, Y $\times$ 6, X $\times$ 6**

Live demo: order matters

```
rotation = Quaternion.from_axis_angle(  
    [1, 0, 0],  
    ROTATION_STEP  
)  
  
args.state.orientation =  
    (rotation * args.state.orientation).normalized
```

Run **R, X×6, Y×6**.

Reset.

Run **R, Y×6, X×6**.

What the audience sees

The same two rotations.

The same two angles.

The same box.

But a different final orientation.

That is:

$$q_y q_x \neq q_x q_y$$

Where this goes next: SLERP

Choose a target orientation (future work, not in the quick demo):

```
target = Quaternion.from_axis_angle(  
    random_axis,  
    random_angle  
)
```

Then smoothly move towards it:

```
orientation = Quaternion.slerp(  
    start,  
    target,  
    progress  
)
```

What is SLERP?

SLERP means:

| spherical linear interpolation

Unit quaternions live on a sphere in four dimensions.

SLERP follows a natural curved path between orientations.

On screen, the motion looks smooth and deliberate.

The double-cover surprise

Two quaternions can represent the same rotation:

$$q \quad \text{and} \quad -q$$

For rendering, nothing changes.

For interpolation, it matters.

We usually choose the shorter path.

Beyond the box

A box is good for teaching.

Future versions could use something more memorable:

- a wireframe ruby gemstone;
- a tiny construction crane;
- a constellation of rotating boxes;
- RUBY letters arranged in 3D;
- a box with local coordinate axes and trails.

Construction-site callback

A crane could give a future talk a nice loop:

Michael began with 3D construction-site visualisation.

We could return with a tiny Ruby-made 3D scene.

Possible parts:

- rotating base;
- boom angle;
- hanging hook;
- camera orbit;
- quaternion-controlled orientation.

What DragonRuby is doing

```
module Main
  def tick(args)
    args.state.demo ||= QuaternionDemo.new
    args.state.demo.tick(args)
  end
end
```

That is enough.

`QuaternionDemo` owns the input, state updates, and drawing.

The bigger lesson

Each extension gave us something:

$$\mathbb{N} \rightarrow \mathbb{Z} \rightarrow \mathbb{Q} \rightarrow \mathbb{R} \rightarrow \mathbb{C} \rightarrow \mathbb{H}$$

- negatives
- fractions
- continuous lengths
- polynomial roots and plane rotations
- 3D orientation

The price of power

Complex numbers preserve commutativity.

Quaternions do not.

But that loss is not arbitrary.

It matches the world:

- 3D rotations do not commute.

Closing thought

The four components of a quaternion are not four angles.

They are not four dimensions we need to visualise.

Together, they describe **one orientation**.

And with one compact formula:

$$p' = qpq^{-1}$$

we can rotate a whole 3D world.

Quaternions are strange numbers

But the geometry they describe is the geometry we already live in.